

Migrating to Oracle Redwood: What Actually Changes in Fusion

Redwood is not a new coat of paint on the classic pages — it's a different page technology, built on Visual Builder Studio and Oracle JET instead of ADF. That distinction is why the migration comes with real functional gains, and real rework, not just a redesign.

10 MIN READ · SUPPLY CHAIN CAPABILITIES

IN THIS DOCUMENT

1. What Redwood actually is
2. Classic vs. Redwood, side by side
3. What you get that classic doesn't have
4. What doesn't change
5. Planning the migration
6. Lessons from the field
7. How Insu Dynamics approaches this

What Redwood actually is

It's easy to assume Redwood is a theme — Oracle does let you apply Redwood styling to classic pages through the Appearance work area, and a themed page is not the same as a rebuilt one. A true Redwood page changes the underlying markup itself, which is also why it can carry functionality a themed classic page simply cannot.

Oracle has been rolling Redwood pages out module by module since 2023, and has signaled that classic pages will eventually be retired outright — starting with SCM, with Financials to follow. Each Fusion environment updates four times a year on a fixed cycle, and Redwood lands inside that cycle screen by screen rather than as a single cutover: new Redwood pages typically appear alongside the classic version, controlled by a profile option, before eventually becoming the only option.

NUGGET

Because the rollout is gradual, you can often opt in to a Redwood page months before it becomes mandatory. Turning it on early, module by module, is a far cheaper way to absorb the change than waiting for Oracle to switch it on for you.

Classic vs. Redwood, side by side

The differences are easiest to see on an ordinary transaction screen — take something as routine as creating a sales order.

Classic ADF Page

Create Sales Order

Every field visible on load — nothing hidden

Shipping & payment shown inline, further down

Cancel

Save

No filters, no saved views —
search by exact attribute to begin

Redwood Page

Customer

Status

Line Status

Smart Filters — orders load immediately

Key fields only
Item, quantity, requested date

▼ Additional Details (collapsed)

Shipping & payment tucked behind the drawer

Lines

Action Status

Tab switcher — no separate page navigation

...

Save

Close/Cancel moves inside "More Actions"
Layout adapts to desktop, tablet, or phone

Illustrative comparison — not an actual product screenshot.

Individually, each change is small. At the scale of a full implementation, they add up to real retraining and retesting — which is exactly why this is a planned migration, not a settings toggle.

What you get that classic doesn't have

These aren't cosmetic wins. Several of the following simply don't exist on the classic side, at any configuration.

Embedded AI, inline with the transaction

Generative AI drafts item and product descriptions, planner notes, and exception summaries directly inside the page. Oracle also ships prebuilt AI agents for specific jobs — helping a service rep process a return, guiding a buyer through procurement policy, or walking a maintenance technician through a repair — and teams that want their own can build agents with AI Agent Studio, a no-code tool. None of this exists on a classic page at any level of personalization; it's Redwood-only.

Planning workspaces built for exploration, not just entry

In Supply Chain Planning, Redwood introduces flexible workspaces where a planner can drag and drop visualizations, place two demand scenarios side by side, and apply AI-driven analytics directly against the numbers in view — a working style the classic planning screens weren't built to support.

Smart Filters and saved, shareable views

Classic search screens generally require an exact-match query before showing anything. Redwood pages like Sales Orders load populated immediately, with filter chips for attributes like customer, order status, or line status layered on top — closer to how a planner or CSR actually works through a queue.

Mobile-capable transactions

Because Redwood is responsive by design, transactions that were desktop-only on classic — creating an interorganization transfer, inspecting received goods — are now available on a phone or tablet, useful anywhere the transaction actually happens: the warehouse floor, not just a desk.

TIP

Lead with the AI and mobile capabilities when building the business case for an early move to Redwood — they're the gains stakeholders notice fastest, and they don't exist on the classic side no matter how the page is configured.

What doesn't change

This is a UI and extensibility migration, not a re-implementation. Your chart of accounts, item masters, ledgers, and integrations stay exactly where they are — Redwood changes how users reach the data, not the data itself.

CAVEAT

Personalizations don't carry over automatically. Page Composer rules and layout changes built for classic ADF pages have to be individually reviewed and rebuilt for Redwood — some won't survive the move as-is, and the longer a customization sits untouched, the more expensive it gets to migrate later.

Planning the migration

1

Inventory what you have

Catalog every classic personalization, Page Composer rule, and custom extension per module, and flag which ones are still actually used.

2

Turn on Redwood early, module by module

Use the profile option to opt in ahead of the mandatory switch, starting with lower-risk modules, while classic remains available as a fallback.

3

Rebuild extensions in Visual Builder Studio

Extensions built through JDeveloper-era ADF tooling need to be rebuilt visually in VBS against Oracle JET — this is genuinely new tooling for most EBS- and classic-Fusion-trained teams.

4

Retrain on the interaction pattern, not just the screen

Progressive disclosure, the "More Actions" menu, and tab switchers change muscle memory even for experienced users — treat this as a real training item, not a five-minute walkthrough.

5

Retest before the mandatory cutover

Automated test suites built against classic ADF markup don't carry over to Redwood's different rendering framework; scripts need to be rebuilt, not just re-pointed.

Lessons from the field

Teams that treat this as "just a new UI" consistently underestimate it. The pages that look simpler on screen are often hiding more capability, not less — and the migration effort tracks how much classic-side customization has accumulated over the years, not how big the company is.

INSU DYNAMICS

How we approach this

Every friction point in a Redwood migration shows up in a predictable place. Here's how we handle each one, point by point.

Personalizations that quietly break when a page moves to Redwood

We inventory every Page Composer rule before migration and classify each as portable, rebuildable, or obsolete — nothing gets rebuilt blind.

Extensions still written for JDeveloper-era ADF tooling

We rebuild extensions natively in Visual Builder Studio, against Oracle JET, so they're supported going forward instead of patched around.

Users retrained on the screen, not the new interaction pattern

We train on the pattern — progressive disclosure, the "More Actions" menu, tab switchers — so muscle memory transfers across every Redwood page, not just the one demoed.

Automated test suites built against classic ADF markup

We rebuild test scripts against Redwood's rendering framework ahead of the mandatory cutover, instead of discovering they're broken at go-live.

Waiting for Oracle's mandatory switch to force the change

We help you opt in early, module by module, using the profile option to move on your own timeline while classic stays available as a fallback.

This is what **innovation drives sustainability** means for us in practice — closing the specific gaps that make a Redwood migration slip, one point at a time, rather than treating it as a UI refresh.